

Smoothing on Dynamic Concurrency Throttling

Janaina Schwarzrock*, Hiago Mayk G. de A. Rocha*,
Arthur F. Lorenzon† and Antonio Carlos S. Beck*

*Institute of Informatics - Federal University of Rio Grande do Sul, Brazil

†Laboratory of Optimization Systems - Federal University of Pampa, Brazil

E-mail: {jschwarzrock, hmgrocha, caco}@inf.ufrgs.br, aflorenzon@unipampa.edu.br

Abstract—Technology scaling has been allowing a growing number of cores in processors to satisfy the increasing demand of new applications, which need to process huge amounts of data in High-Performance Computing (HPC). However, considering that many parallel applications have limited scalability, not always activating the maximum number of available cores to execute an application will provide the best outcome in energy and performance (represented by the Energy-Delay Product, or EDP). Because of that, many works have already proposed different Dynamic Concurrency Throttling (DCT) techniques to adapt thread count as the application executes. However, dynamically tuning (i.e. increasing or decreasing) thread count implies in overheads related to caches warm-up and thread reallocation across the cores. This overhead may become very significant when thread count changes often during execution, which may overcome the benefits brought by DCT. This problem is further aggravated in Non-Uniform Memory Access (NUMA) systems since some cores are more distant, in terms of latency, than others. With that in mind, in this paper, we propose a smoothing-based strategy to minimize the thread count changes and, consequently, mitigate the aforementioned overhead. Our proposal is generic and aims further to improve the optimization results of any DCT technique. As case-study, we performed experiments on two multicore systems with nine well-known benchmarks, showing that our smoothing technique improves EDP results of offline and online state-of-the-art DCT techniques by up to 93% and 89% (both 22% on mean), respectively.

Index Terms—dynamic concurrency throttling, data smoothing, EDP optimization.

I. INTRODUCTION

Saving energy has become one of the most essential priorities in the last few decades [1]. Specialists predict that Information and Communication Technology (ICT) will be responsible for about 20% of the global electricity demand in 2030, and data centers will take a considerable slice of it [2]. At the same time, advancements in modern high-performance computing (HPC) environments have been promoting enhancements in several domains, such as medicine, bioinformatics, artificial intelligence, weather, and stock exchange forecasts. In this scenario, Thread-Level Parallelism (TLP) has been widely exploited to accelerate these applications using all available resources in the modern multicore processors.

Nonetheless, several hardware and software aspects limit the TLP scalability, such as off-chip bus saturation and data synchronization [3]. It means that activating the maximum number of available threads to execute a parallel application

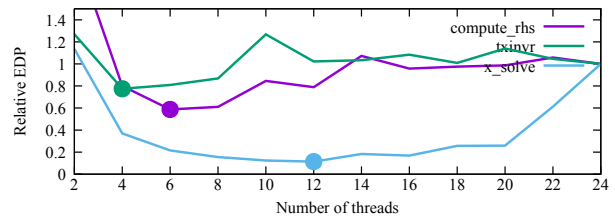


Fig. 1. EDP of three different parallel regions of the SP benchmark considering different number of threads (*the lower, the better*). The lowest EDP for *compute_rhs*, *txinvr* and *x_solve* is achieved with 6, 4 and 12 threads, respectively.

– the standard choice for most software developers [4]–[7] – will seldom provide the best energy-saving and performance trade-off, represented by the Energy-Delay Product (EDP). Moreover, the ideal thread count may change according to the system (e.g., microarchitecture, number of cores, available memories), application, and input set [8], [9]. Therefore, the task of rightly choosing the ideal configuration of thread count to deliver the lowest EDP is not straightforward.

On top of that, a parallel application may comprise many parallel regions, and each one may have a different configuration of thread count that minimizes EDP. Fig. 1 illustrates this scenario highlighting with bullets the number of threads (*x-axis*) that delivers the best EDP (*y-axis*) for three different parallel regions of the Scalar Pentadiagonal (SP) kernel [10] in a 12-core 2-way Simultaneous Multithreading (SMT) Intel machine. We normalized the results by the standard parallel execution, which uses the maximum number of available hardware threads (24 threads, in this machine). These results illustrate that parallel regions of the same application may differ on the thread count configuration that delivers the lowest EDP.

Because of that, many works have already proposed different **Dynamic Concurrency Throttling (DCT)** techniques to automatically learn the ideal thread count and adjust them as the application executes [3]–[8], [11]–[18]. However, when applying DCT, the transition from one parallel region to another will in many cases generate an overhead related to different hardware characteristics, such as cache warm-up and the number of Translation Lookaside Buffer (TLB) misses, which are a result of thread reallocation across the

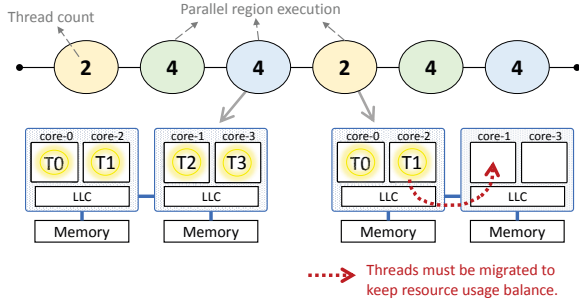


Fig. 2. Each circle represents an execution of a parallel region and the ones with the same color represent the same region. The number inside the circle show the thread count at that moment of the execution. In this example, threads are placed in a contiguous way on a system with 2 NUMA nodes, 2 cores per node. Depending on the changes in the the thread count, threads will migrate.

available cores, since the thread count has changed. This overhead further increases in Non-Uniform Memory Access (NUMA) machines since the latency will be heterogeneous depending on the group of cores (i.e., the NUMA node). Such overhead, when not controlled, may have a huge impact on the gains brought by DCT, significantly amortizing the gains of the technique. However, it has been ignored by most of the previous works. Fig. 2 illustrates the above discussion, showing what happens when the thread count changes from 4 to 2. For instance, if threads are allocated in a contiguous way, one thread (T1) will migrate from one NUMA node to another after T2 and T3 are destroyed, causing a significant impact on performance and energy consumption.

In this work, we show that minimizing the thread count changes alleviates the overhead mentioned above and, consequently, improves the results of the DCT techniques. However, the challenge lies in minimizing such changes to a point where the resulting gains of DCT are not significantly impacted. With that in mind, we propose a strategy to reduce the thread count changes, which can be used by any existing DCT approaches to enhance their results further. Our strategy is based on the data smoothing technique, which is a statistical approach used to remove noises in data analysis [19]. For that, we treat the number of threads used during the application execution as a time series and apply the Weighted Moving Average (WMA) to smooth the thread count changes.

We propose two different algorithm variations for our smoothing strategy: WMA – *right-aligned* and *centered* – and develop them to work with *offline* and *online* DCT techniques. We evaluate our strategy in two multicore systems executing nine well-known benchmarks. Considering the overall geometric mean, our smoothing technique improves EDP results of offline (up to 93% and 22% on mean) and online (up to 89% and 22% on mean) state-of-the-art DCT techniques, with low cost (4.5% on mean).

Therefore, the contributions of this paper include:

- We show the overhead of switching the thread count configuration as the application executes, which is intrinsic to any DCT technique. We also reveal that, on NUMA systems, such cost increases;

- We propose a technique based on data smoothing to minimize thread count changes and, consequently, reduce the switching overhead. For that, we consider and evaluate two variations of the WMA smoothing technique – *right-aligned* and *centered* –, and also propose *offline* and *online* algorithms to be integrated to different DCT approaches;
- As a case study, we show how our smoothing strategy works when applied to Hoder [7], an state-of-the-art online learning DCT technique.

The remainder of this paper is organized as follows. Section II discusses the background and related work. Section III explains the data smoothing technique we have used and how we have adapted it to our problem. Section V describes the experimental methodology, and Section VI presents the evaluation results. Section VII presents a case study to show how our smoothing technique works on an online learning DCT library. Section VIII draws the final considerations.

II. BACKGROUND AND RELATED WORK

Concurrency throttling, also known as *Thread throttling*, is an optimization technique that manages the number of threads used to run a parallel application. Fig. 3 illustrates the different way it can be applied.

The number of threads can be adjusted in a **Static** or **Dynamic** way. *Static thread throttling* consists of setting an optimized thread count configuration at the beginning of the execution, keeping it until the application ends (Fig. 3a). That is, it uses the same thread count for all parallel regions. The Dynamic approach, also known as *Dynamic Concurrency Throttling* (DCT), considers the different parallel regions' behavior by changing the number of threads as the application executes (Fig. 3b-c). That is, DCT techniques have a set of optimized thread count configurations, one for each parallel region.

For simplicity, when we use the term “*solution*” for static approaches, we refer to a single thread count configuration t^* that optimizes the whole application; and when it comes to dynamic approaches, we use “*solution*” to refer to a set of thread count configurations $\mathcal{B} = (t_1^*, \dots, t_k^*)$, which has one optimized number of threads t_j^* for each parallel region j of the application. Thread Throttling approaches also differ on when they learn for the best solution: **Offline** ones decide the best solution beforehand application execution (Fig. 3a-b), while **Online** ones converges to a solution at runtime (Fig. 3c), during execution.

A. Static and Offline Concurrency Throttling

Some works perform static optimization, which also requires an offline learning process. They select a single solution t^* that must be known beforehand the execution. Therefore, they require re-executing the whole application after finding the best solution; and re-executing the learning process in case there are any changes in the execution of the application, to exploit the the advantages of the technique at this maximum. The approaches proposed in [20] and [21] are examples of

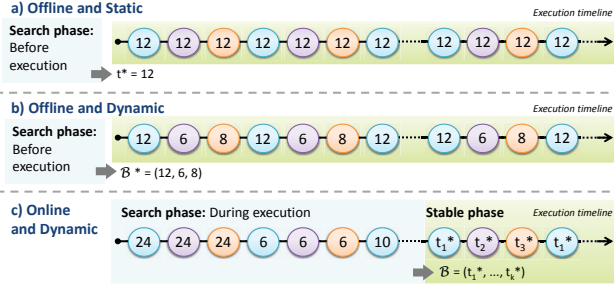


Fig. 3. Thread throttling approaches. **Offline approaches** decide the best thread count beforehand the execution. They can be either: *Static* – a single fixed thread count configuration t^* – or *Dynamic* – a set of thread count configuration $B = (t_1^*, \dots, t_k^*)$. **Online approaches** learn the best solution at runtime; therefore they **are always dynamic**.

static thread throttling techniques. The approach in [20] seeks t^* by running the application multiple times for a short period with different thread count configurations. It uses OS-level information to guide the search. The approach in [21] predicts t^* using a multiple linear regression model.

B. (Offline or Online) Dynamic Concurrency Throttling (DCT)

Most of the state-of-art are DCT. They select a particular thread count configuration for each parallel region to consider their different requirements.

Offline DCT techniques. The following strategies find for a solution before the application's execution. The authors in [5] proposed a model to predict, for each parallel region, the number of threads to run on each NUMA node to maximize bandwidth usage while minimizing the working cores. The authors in [6] proposed an offline search approach to find the best configuration of thread count and thread and data mapping.

Online DCT techniques. The following techniques find for a solution during application execution. They differ mainly on how to learn/decide the ideal thread count configuration for each parallel region. The dynamic compiler in [4] reduces the active threads by combining them to minimize communication and synchronization. Based on runtime observations, the technique in [8] decreases the number of active threads to avoid cache saturation or whenever a thread is suspended. Some approaches [3], [12]–[15], [17] use prediction models to estimate the best configuration. For that, during the first few executions of each parallel region they collect hardware performance counters to serve as input for their model. Other works evaluate several configurations employing heuristics to converge to an ideal configuration. The authors in [11], [16], [18] proposed different search algorithms based on hill-climbing heuristic, and the library in [7] uses a Fibonacci-based algorithm. The study by [22] investigated the data migration impacts on DCT techniques on NUMA systems.

C. Alleviating the DCT overhead.

Most of the previous DCT techniques have ignored the switching overhead. Only the authors in [6] have made some effort to alleviate it. However, their approach for minimizing

thread count configuration changes requires a complex process of analyzing a application's memory access profile to identify parallel regions that share data, which implies in a high implementation overhead for the developer or user, precluding it of being generic. Instead, our smoothing technique is simple and lightweight. In this way, our proposal is orthogonal to any of the existing DCT approaches, either offline or online, being easy to integrate to any of them given its nature.

III. SMOOTHING THE THREAD COUNT CHANGES

Problem Definition. A parallel application is composed of a set $\mathcal{R} = \{r_1, \dots, r_k\}$ of k parallel regions. Each parallel region $r_j \in \mathcal{R}$ executes with a thread count configuration $t \in [1, \dots, n]$, where n is the maximum number of available hardware threads. Therefore, a complete set of configuration for all k parallel regions is given by $\mathcal{C} = (t_1, \dots, t_k)$, where t_j is the thread count configuration for the parallel region r_j . Then, the objective is to find the optimal $\mathcal{C}^*$, which minimizes the EDP of the whole application.

Finding $\mathcal{C}^*$ is a problem of time complexity $\mathcal{O}(n^k)$. Depending on the value of k and n , an exhaustive search to find $\mathcal{C}^*$ is unfeasible [23]. Therefore, instead of finding the optimal solution $\mathcal{C}^*$, one can seek the best thread count configuration t^* for each parallel region *individually*, which means having a set $B = (t_1^*, \dots, t_k^*)$, where t_j^* minimizes EDP of the parallel region r_j .

As previously discussed in Section II, finding B is a common approach of DCT techniques. However, the cost of switching the thread count from one region to another as the application runs can make B a sub-optimal solution. Hence, we refer to B as a *best-effort dynamic solution*.

In this work, we aim to alleviate the switching overhead to improve B 's results. That is, we want to enhance DCT techniques. For that, we propose a strategy to smooth the number of threads that comprise B in order to reduce and smooth the configuration changes that it impose. More precisely, we treat the execution of a parallel application, with solution B , as time series (a time series composed of the thread count configuration used on each execution of a parallel region) and apply a data-smoothing technique.

Next, we explain the smoothing technique we have applied in our strategy and how we adapt it to our purpose. For simplicity, Table I summarizes all the variables in our strategy.

Smoothing technique overview. Our strategy is based on the Weighted Moving Average (WMA), a lightweight but also powerful smoothing technique. Being lightweight, one can include our strategy in any DCT technique with a negligible overhead. Next, we explain our proposal.

Given a time series $y_1, \dots, y_m$ and a weight w_i for each y_i , the WMA technique creates a new time series $\bar{y}_1, \dots, \bar{y}_m$, where each smoothed data point $\bar{y}_i$ is taken by averaging neighboring weighted data points in a window of n data points, as in:

$$\bar{y}_i = \frac{(y_i w_i) + (y_{i-1} w_{i-1}) + \dots + (y_{i-n-1} w_{i-n-1})}{w_i + w_{i-1} + \dots + w_{i-n-1}} \quad (1)$$

TABLE I
SUMMARY OF VARIABLES' MEANING

Variable	Meaning
k	Number of parallel regions in an application
$\mathcal{B} = (t_1^*, \dots, t_k^*)$	t_j^* is the best #thread for a parallel region j
i (a point)	Execution of a parallel region
m	Number of points in an application's execution
$\mathcal{P} = (p_1, \dots, p_m)$	p_i is the parallel region that runs at point i
$\mathcal{Y} = (y_1, \dots, y_m)$	y_i is the number of active threads at point i
$\mathcal{E} = (e_1, \dots, e_m)$	e_i is the execution time of the point i
W	Time window
pW	Partial time window (left- or right-side window)
$\mathcal{Q} = (q_1, \dots, q_k)$	q_j is the execution time reference of the parallel region j

In our strategy, y_i is the number of running threads at the point i , where a point is a parallel region execution (represented by a circle in Fig. 2), and the weight w_i is the i 's execution time. It is fair to give different weights to each y since the configurations that are enabled only in a short period tend not to amortize the switching overhead.

In this work, we experiment with two WMA's variations:

- **WMA-right-aligned.** It is the one presented in Equation 1, where the current point i is the rightmost end of the window. E.g., to get $\bar{y}_i$ with a window of three points, it considers the data points y_{i-2} , y_{i-1} , and y_i .
- **WMA-centered.** Slightly different from the previous one: in this variation, the current point i is the center of the window. E.g., to get $\bar{y}_i$ with a window of three points, it considers the data points y_{i-1} , y_i , y_{i+1} .

For each variation, we proposed algorithms to work either *offline* or *online*. Next, we first describe our offline algorithms (Section III-A). Then, we explain how we adapt them to work online (Section III-B).

A. Offline smoothing

Overview. As described above, to smooth thread count changes, we need the time series $\mathcal{Y} = (y_1, \dots, y_m)$, where y_i is the thread count at the point i . We consider a point i as a parallel region execution. There are m points in a parallel application's execution. We also need the set $\mathcal{E} = (e_1, \dots, e_m)$, where e_i is the execution time of the point i . The set $\mathcal{E}$ comprises the weights on our technique. After having the time series $\mathcal{Y}$ and the weights $\mathcal{E}$, we smooth $\mathcal{Y}$, resulting in the smoothed time series $\bar{\mathcal{Y}} = (\bar{y}_1, \dots, \bar{y}_m)$. Finally, we run the application using $\bar{\mathcal{Y}}$.

Getting data points $\mathcal{Y}$ and weights $\mathcal{E}$. To gather the data points and weights, We executed the application once with the maximum number of threads (notice that this is the process for our offline technique). At each point i , we collect the parallel region p_i and the execution time e_i ¹. After the execution ends, we have $\mathcal{P} = (p_1, \dots, p_m)$ and $\mathcal{E}$. Then, we combine $\mathcal{P}$ with a dynamic solution $\mathcal{B}$, which gives the best thread count t^* for

¹We collect p_i with the variable `ptr_region` of the `libgomp` and e_i with the OpenMP function `omp_get_wtime`.

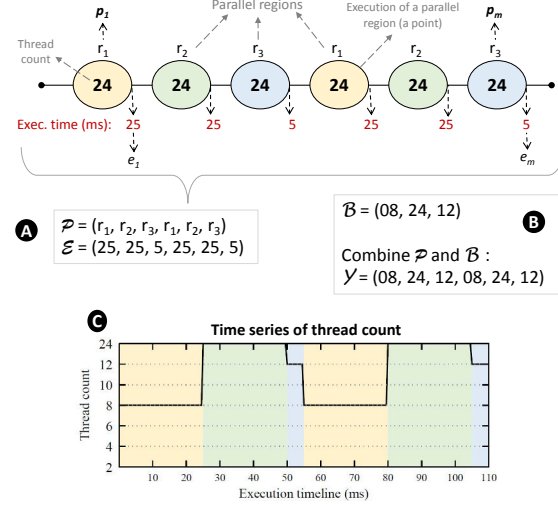


Fig. 4. The offline process of creating the time series of thread count. (A) Execute the application once with the maximum number of threads to collect parallel region sequence execution $\mathcal{P}$ and their execution time $\mathcal{E}$; (B) Creating $\mathcal{Y}$ by combining $\mathcal{P}$ and $\mathcal{B}$. (C) Graphic representation of the time series when considering $\mathcal{Y}$ and $\mathcal{E}$.

each parallel region, to create $\mathcal{Y}$: for each $p_i \in \mathcal{P}$, $y_i = t_{p_i}^*$. That is, each $y_i \in \mathcal{Y}$ is the thread count at the point i when the application runs with $\mathcal{B}$.

As aforementioned, $\mathcal{B}$ is a best-effort dynamic solution. How to get $\mathcal{B}$ is not stated by our technique. There are already many techniques to get it (as discussed in Section II); One can use any of them. We will explain how we have collected $\mathcal{B}$ for our experiments in Section IV.

Notice that to collect $\mathcal{E}$, we execute the application with the maximum number of threads instead of the $\mathcal{B}$ solution. Running the application with $\mathcal{B}$ can distort the relative time of each parallel region because of the switching overhead and, consequently, the weight for each point could be wrong. Therefore, we use the same number of threads – the maximum one – for all parallel regions while collecting the weights.

To illustrate the process of getting $\mathcal{Y}$ and $\mathcal{E}$, we will use a toy example (see Fig. 4). Consider an application with three parallel regions (r_1, r_2, r_3) running on a 12-core 2-way SMT machine, which has 24 hardware threads in total. In this application, the parallel regions are executed twice in that exactly same order; Hence, $m = 6$ and $\mathcal{P} = (r_1, r_2, r_3, r_1, r_2, r_3)$. To collect $\mathcal{P}$ and $\mathcal{E}$ we execute the application once with the maximum number of threads (Fig. 4A)². Consider that the best-effort dynamic solution for this application is $\mathcal{B} = (8, 24, 12)$; Hence, $\mathcal{Y} = (8, 24, 12, 8, 24, 12)$ (Fig. 4B). Fig. 4C illustrates the time series of our example, which combines $\mathcal{Y}$ and $\mathcal{E}$.

Next, we first explain the variation *WMA-right-aligned*, and then *WMA-centered*.

²For simplicity, in our example, the execution time is the same for any execution of the same parallel region (e.g., p_1 and p_4 refers to a execution of the parallel region r_1 and, in the both points, 1 and 4, r_1 lasts the same time, i.e., $e_1 = e_4 = 25$ ms; However, in actual applications a region r_i may have different execution time at each point, which means that, although $p_h = p_j = r_i$, $e_h \neq e_j$).

Algorithm 1 Offline WMA-right-aligned

Require: $\mathcal{B}$, $\mathcal{P}$, and $\mathcal{E}$.**Ensure:** $\bar{\mathcal{Y}} = \{\bar{y}_1, \dots, \bar{y}_m\}$.

```
1:  $\mathcal{Y} \leftarrow \forall p_i \in \mathcal{P}, y_i = t_{p_i}^*$ ;
2: for  $i = 1$  to  $m$  do
3:    $pW \leftarrow (W - e_i)$ ;
4:    $ywLeft, wLeft \leftarrow \text{Alg. 2}(\mathcal{Y}, \mathcal{E}, i, pW)$ ; /* Left */
5:    $\text{sumYW} \leftarrow (y_i \times e_i) + ywLeft$ ;
6:    $\text{sumW} \leftarrow y_i + wLeft$ ;
7:    $\bar{y}_i \leftarrow \text{round}(\text{sumYW}/\text{sumW})$ ;
8: end for
9: return  $\bar{\mathcal{Y}}$ ;
```

Algorithm 2 Compute the right or left side of the window

Require: $\mathcal{Y}$, $\mathcal{E}$, i , pW .**Ensure:** yw and w .

```
1:  $yw, w \leftarrow 0$ ; /* Left side: */
2: for  $j$  from  $(i - 1)$  to  $1$ ; do
   /* Right side: */
3: for  $j = (i + 1)$  to  $m$ ; do
4:   if  $(w + e_j) > pW$  then
5:      $yw \leftarrow yw + (y_j \times (pW - w))$ ;
6:      $w \leftarrow pW$ ;
7:   else
8:      $yw \leftarrow yw + (y_j \times e_j)$ ;
9:      $w \leftarrow w + e_j$ ;
10:  end if
11:  if  $w = pW$  then
12:    break;
13:  end if
14: end for
15: return  $yw$  and  $w$ ;
```

Smoothing thread count with WMA-right-aligned. Algorithm 1 describes our offline WMA-right-aligned technique. It first creates the time series $\mathcal{Y}$ using $\mathcal{P}$ and $\mathcal{B}$ (line 1). Then, it smooths each thread count $y_i \in \mathcal{Y}$ (lines 2-8). For that, our algorithm implements the equation 1 (lines 5-7), considering the points (i.e., the parallel regions executions) inside a time window W , including the current one i , which is the rightmost one. Notice that when i takes long enough ($e_i \geq W$), changing the thread count is worth it; In that case, our algorithm does not consider any other region since it will have no remaining window ($pW \leq 0$, line 3). Otherwise, it also takes into account the left-side points (Algorithm 2).

Algorithm 2 sums up the weighted points y of the left-side ($ywLeft \leftarrow yw$) and their weights e ($wLeft \leftarrow w$). Notice that $w \leq pW$ (lines 4 and 11). That means our algorithm can partially consider the leftmost point j 's time (lines 4-6, if its time e_j were longer than the remaining window $pW - w$), or when there are no regions enough to reach the window, it averages data proportionally.

Fig. 5 uses our previous toy example to show how our WMA-right-aligned algorithm works. More precisely, it illustrates when $i = 2$. To ensure that the smoothed value $\bar{y}$ is

Algorithm 3 Offline WMA-centered

Require: $\mathcal{P}$, $\mathcal{E}$, and $\mathcal{B}$.**Ensure:** $\bar{\mathcal{Y}} = \{\bar{y}_1, \dots, \bar{y}_m\}$.

```
1:  $\mathcal{Y} \leftarrow \forall p_i \in \mathcal{P}, y_i = b_{p_i}$ ;
2: for  $i = 1$  to  $m$  do
3:    $pW \leftarrow (W - e_i)/2.0$ ;
4:    $ywLeft, wLeft \leftarrow \text{Alg. 2}(\mathcal{Y}, \mathcal{E}, i, pW)$ ; /* Left */
5:    $ywRight, wRight \leftarrow \text{Alg. 2}(\mathcal{Y}, \mathcal{E}, i, pW)$ ; /* Right */
6:    $\text{sumYW} \leftarrow ywLeft + (y_i \times e_i) + ywRight$ ;
7:    $\text{sumW} \leftarrow wLeft + e_i + wRight$ ;
8:    $\bar{y}_i \leftarrow \text{round}(\text{sumYW}/\text{sumW})$ ;
9: end for
10: return  $\bar{\mathcal{Y}}$ ;
```

inside the thread count search space, our algorithm uses *index values* instead of *raw values* of thread count. Therefore, in this example, instead of $\mathcal{Y} = (8, 24, 12, 8, 24, 12)$, $\mathcal{Y} = (4, 7, 6, 4, 7, 6)$ – See Section IV (Table II) for more detail about our search space.

Smoothing thread count with WMA-centered. Algorithm 3 describes our offline WMA-centered technique. It is very similar to the WMA-right-aligned one, but when it smooths y_i , i is the center point of the window. Our WMA-centered uses the Algorithm 2 to compute the left side and the right side. Left and right sides consider half of the remaining window (discounting from W the current point's time e_i). Fig. 6 illustrates how WMA-centered smooths the point i when $i = 2$.

Smoothed solution. After running the offline smoothing algorithm, either WMA-right-aligned or WMA-centered, we have $\bar{\mathcal{Y}}$, which has the smoothed thread count for each parallel region execution. Then, we run the application using $\bar{\mathcal{Y}}$. As the results section will show, our offline smoothing algorithms significantly reduce the thread count changes and improve the $\mathcal{B}$ solution's results.

Offline smoothing implementation. We implemented our offline smoothing algorithms in Python. To run the application with the smoothed time series $\bar{\mathcal{Y}}$, we implemented a library that reads a file with $\bar{\mathcal{Y}}$, stores the data in a vector, and uses the proper configuration at each parallel region execution. We implemented that library in C and included it in the GCC OpenMP runtime library, *libgomp*.

B. Online smoothing

Overview. The online smoothing strategy follows the same idea of the offline one. However, since it smooths the thread count changes at runtime, it collects $\mathcal{P}$ and $\mathcal{E}$ as the application executes, and smooths y_i only when the application's execution reaches a point i . Then, it sets the smoothed $\bar{y}_i$ as the thread count to run p_i .

Online WMA-right-aligned. Whenever the application's execution reaches a point i , it knows p_i , i.e., the parallel region that is going to run, and stores it in $\mathcal{P}$. However, at this moment, it does not know e_i (i.e., the i 's execution time) yet since e_i is known only when p_i ends. Therefore, the first time a parallel region runs, it runs with the maximum number of

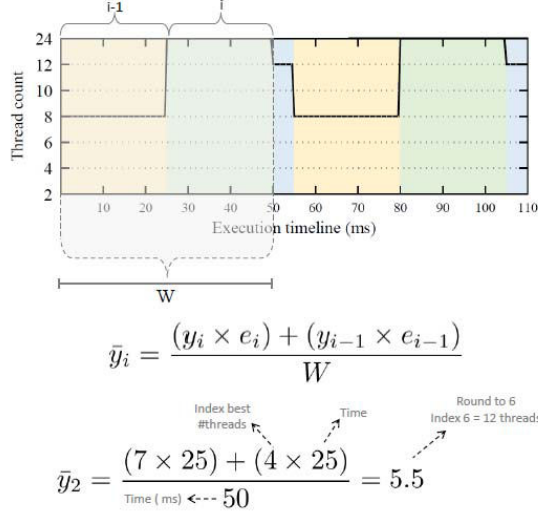


Fig. 5. Illustration of our WMA-right-aligned algorithm smoothing the thread count used to run a parallel region at point 2 on our toy example.

threads, and our algorithm collects and stores its execution time. It is done only at the first execution of each parallel region. In this way, instead of having $\mathcal{E} = (e_1, \dots, e_m)$, which has the execution time of each point i , our online technique has $\mathcal{Q} = (q_1, \dots, q_k)$, which gives the first execution time of each parallel region. In that way, in our online algorithm, instead of e_i , the weight of y_i is q_{p_i} , where p_i is the parallel region that run at the point i .

Online WMA-centered. As mentioned before, the online smoothing algorithm does not know the whole $\mathcal{P}$ a priori. As the application executes, our algorithm has to feed $\mathcal{P}$ with the parallel regions. Therefore, the $\mathcal{P}$ has only the left side of the window, with no information about the right side, so the online WMA-centered variation has to predict the right side of the window.

For that, we use a simple prediction based on the two-level local branch prediction [24]: given a current parallel region b and its previous one a , we predict the next one c . We implement it as a per-region history vector to store (a, c) , where a is the index and c is the value. In our implementation, a new pair (a, c) always overwrites the previous one. That is, when there is a parallel regions execution sequence like $P = (a, b, c, a, b, d)$, and the execution starts the parallel region d , the previous entry of b 's history vector (a, c) is overwritten by (a, d) .

IV. BEST-EFFORT DYNAMIC SOLUTION - $\mathcal{B}$

In this section, we explain how we have gotten $\mathcal{B}$, the Best-effort dynamic solution, for our experiments. Besides using $\mathcal{B}$ for comparison purpose (as we will explain in Section V), we also use $\mathcal{B}$ as the input for our smoothing algorithms, as already explained in Section III. We followed the same methodology to find the $\mathcal{B}$ solution as [7], and briefly describe it here.

To find $\mathcal{B}$, we executed each application with all thread count configurations inside our search space (we describe it

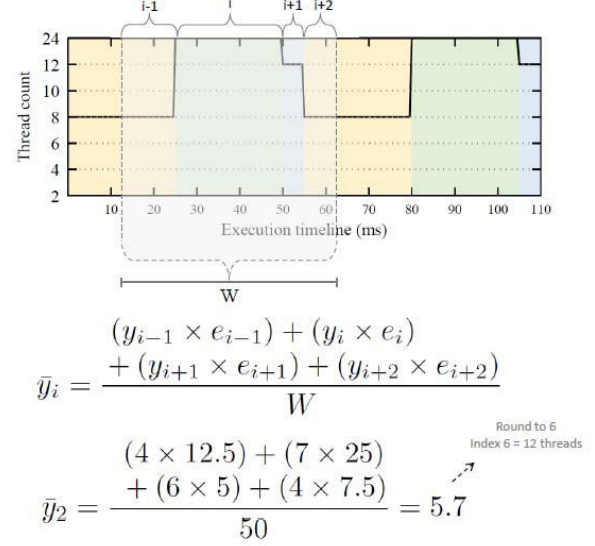


Fig. 6. Illustration of our WMA-centered algorithm smoothing the thread count used to run a parallel region at point 2 on our toy example.

TABLE II
THREAD COUNT SEARCH SPACE

Intel24	Index	1	2	3	4	5	6	7
	#threads	2	4	6	8	10	12	24
Intel88	Index	1	2	3	4	...	22	23
	#threads	2	4	6	8	...	44	88

below). Considering a search space of n configurations, we ran each application n times. During these executions, we collected the EDP of each parallel region individually. Then, we analyzed the EDP results of each parallel region j , and selected the thread count t_j^* that provides the lowest EDP for each region to build $\mathcal{B} = (t_1^*, \dots, t_k^*)$.

The search considers only even thread count configurations and ignores the configurations using SMT except for the one with the maximum number of threads – that is, given a system with H physical cores, each of them composed of x -way SMT, the search space considers the interval of the number of physical cores $[2, H]$ and the maximum number of threads xH (i.e. all the cores with SMT enabled). Table II presents the search space for our two machines.

V. METHODOLOGY

A. Benchmarks and Platforms

Platforms. We performed the experiments on two real NUMA platforms with Linux OS kernel v.4.19.0:

- **Intel24:** 2x 6-core Intel Xeon E5-2630 (Sandy Bridge) @2.3GHz, 2-way SMT (2 nodes / 12 cores / 24 threads). Each core has a 32KB L1 cache and 256KB L2 cache; and shares 2x15MB L3 cache and 2x16 GB of DRAM;
- **Intel88:** 2x 22-core Intel Xeon E5-2699v4 (Broadwell) @2.2 GHz, 2-way SMT (2 nodes / 44 cores / 88 threads).

Each core has a 32KB L1 cache and 256KB L2 cache, and shares 2x55MB L3 cache and 2x128 GB of DRAM;

Benchmarks. We used nine OpenMP parallel applications written in C/C++: **Six kernels** from the NAS Parallel Benchmark [10], [25]: BT, CG, FT, LU, MG, and SP; and **three applications** from different domains: *Fast Fourier Transform* (FFT) [26]; *Jacobi* (JA) [27]; and *Poisson* (PO) [27]. The benchmarks were executed with the following input sets: the kernels from the NAS parallel benchmark with the standard input sets “B” or “C”; FFT with an array of 10000 elements and convergence of 25 iterations; JA with a square matrix of 8192 and 1500 iterations; and PO with a square matrix of 768 and a tolerance of 10^{-6} . With this set, we cover different sorts of behaviors, w.r.t. to Thread-level Parallelism (TLP), Instruction-Level Parallelism (ILP), and amount of memory operations.

B. Set of experiments

We evaluate our smoothing technique by comparing the four algorithm variations – offline WMA-right-aligned, offline WMA-centered, online WMA-right-aligned, and online WMA-centered – with the following.

- **Best-effort dynamic solution:** As explained in Section IV, it is a solution composed of thread count configurations, one for each parallel region – i.e., $\mathcal{B} = (t_1^*, \dots, t_k^*)$, where each t_j^* provides the lowest EDP for the parallel region j . We explain how we get $\mathcal{B}$ in Section IV.
- **Optimization upper-bound:** A single thread count configuration t^* that optimizes the whole application, i.e., every parallel region runs with the same t^* (the number of threads never changes).

When running parallel applications on NUMA systems, thread and data mapping strongly influence the application’s overall performance and energy consumption [28]. Therefore, we consider two executions scenarios:

- (i) Execution with the default thread and data mapping (*Scatter* and *First-Touch* [29]);
- (ii) Execution with the best thread and data mapping policy for each application.

To find the best policies, we have evaluated each application with different thread mapping policies (*Scatter*, *Contiguous*, and *Close* [6], [30]) and data mapping policies (*First-Touch* and *Interleave* [31]). We performed this search on our two machines, always running the applications with the maximum number of threads.

C. Implementation and Execution

To run the applications with a dynamic solution (either $\mathcal{B}$ or a set of smoothed thread count configurations $\bar{\mathcal{Y}}$), where thread count changes and threads migrate during the execution, we modified the GCC OpenMP runtime library (*libgomp*). We set the thread count in the *gomp_parallel_start* function, and use the function *pthread_setaffinity_np* to migrate threads. In all experiments, we configured the data mapping policy beforehand execution. The *First-Touch* policy is already the

default in Linux [32], and for the *Interleave* policy, we have used the Linux *numactl* tool [31]. We experimentally define the time window W of our smoothing algorithm to 50ms.

We used the Intel Running Average Power Limit (RAPL) [33] to collect energy consumption. We consider the energy spent by the entire package (i.e., cores and caches) and the DRAM module. To compute the execution time we used the *omp_get_wtime* function from the OpenMP library. We compiled the applications with GNU GCC/G++ 8.3 and OpenMP 4.5, with the optimization flag -O3. We executed each experiment ten times (standard deviation of $\sim 1\%$).

VI. EXPERIMENTAL RESULTS

Figures 7 and 8 present the EDP results (*y-axis*) for the entire benchmark set along with their geometric mean (*x-axis*) considering the two scenarios (i and ii) on the *Intel24* and *Intel88* machines, respectively. All the results are normalized w.r.t. the standard OpenMP execution, which uses the maximum number of available threads in the system with the *Scatter* thread mapping policy and the Linux’s *First-Touch* data mapping [29], referred to as the **baseline** (values lower than 1 are better than the baseline).

Next, we discuss these results. First, we analyze the results of the best-effort dynamic solution $\mathcal{B}$ by comparing it with the optimization upper-bound to show that dynamic solutions still have room for optimization, which motivates our proposal (Section VI-A). Then, we apply our proposal on $\mathcal{B}$ and show that it can improve a dynamic solution by smoothing thread count changes, reaching EDP results near the upper bound (Section VI-B). Finally, we evaluate the online version of our strategy to show that it is lightweight and delivers similar results to the offline one (Section VI-C).

A. Why does DCT need a smoothing technique?

Best-effort dynamic solutions provided significant EDP gains when compared to the **baseline** regardless of the system. On the average of all benchmark set, EDP was improved by 30% on the *Intel24* and 24% on the *Intel88* on both scenarios (i and ii). However, most of the $\mathcal{B}$ solutions are far from the optimization upper-bound. And, for some cases, $\mathcal{B}$ is worse than the baseline, e.g., CG.B, MG.B, SP.B on *Intel88* (Fig. 8ii).

The cost of changing the number of threads as the application executes causes this gap. This cost is not considered while searching for $\mathcal{B}$ since it finds the best configuration for each parallel region *individually*. Moreover, when the search space increases, it is more likely that the changes increase in both number and intensity. Because of that, the solutions on *Intel88* are 71% worse than the optimization upper-bound, while on *Intel24*, the difference is only 5% when considering the geometric mean of the entire benchmark set on both scenarios (i and ii). Also, notice that on the experiments with the best thread and data mapping policy (scenario ii), where there also have thread migrating, the difference between $\mathcal{B}$ and the optimization upper-bound is $2.2\times$ (Fig 8ii).

When the thread count changes very often in a short period, the benefit of using the best configuration for each parallel

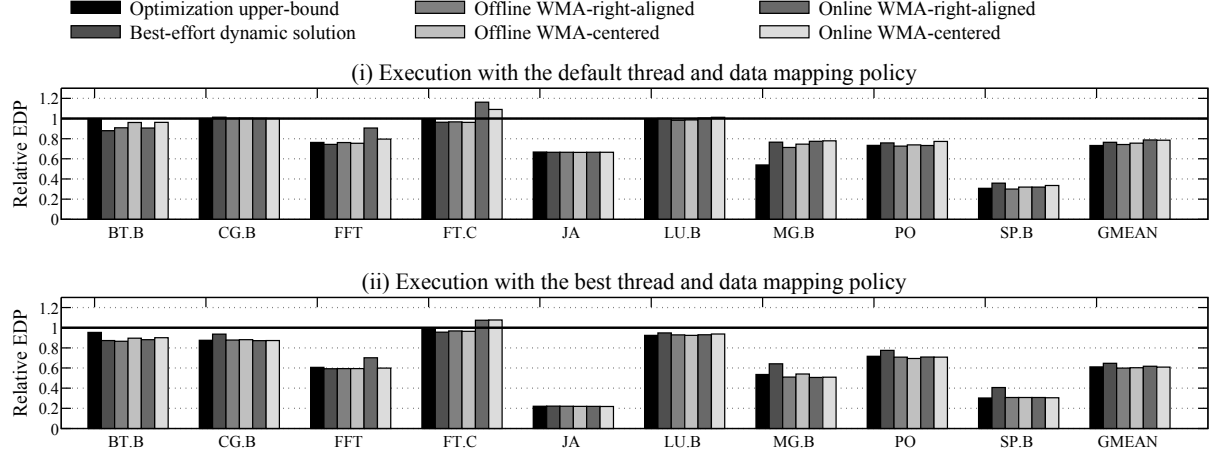


Fig. 7. EDP results on the *Intel24* machine. The results are normalized to the default execution (lower is better).

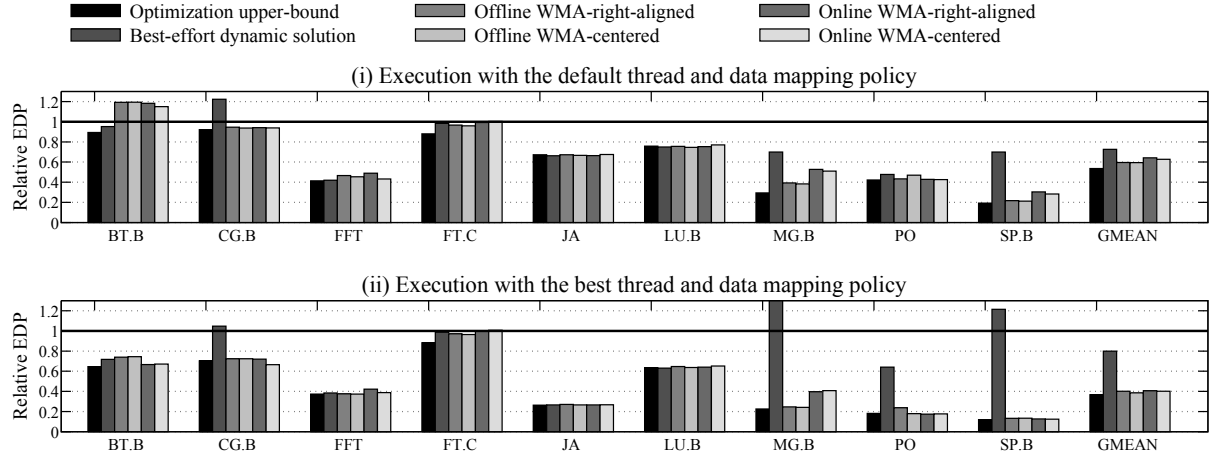


Fig. 8. EDP results on the *Intel88* machine. The results are normalized to the default execution (lower is better).

region may not compensate for the switching cost. To illustrate that, we use two representative examples: BT.B on the *Intel24* machine and MG.B on *Intel88*. The BT.B's $\mathcal{B}$ solution on *Intel24* is better than the optimization upper-bound (Fig. 7-ii), while the MG.B's $\mathcal{B}$ solution on *Intel88* is worse than the baseline (Fig. 8-ii). Fig. 9 shows the thread count (y-axis) used during the first 0.2 seconds of these two benchmarks (x-axis) when they run with $\mathcal{B}$. Notice that MG.B has many changes in a short period (Fig. 9b), while BT.B has few changes (Fig. 9a) – the parallel regions of BT.B take longer, and some neighboring parallel regions have the same best configuration

To minimize the switching overhead and, consequently, improve the results of the solutions, it is necessary to reduce and smooth the thread count changes. Next, we present the results of our data smoothing-based technique applied on $\mathcal{B}$.

B. Offline smoothing results

The algorithm variations – **Offline WMA-right-aligned** and **Offline WMA-centered** – presented very similar optimization

results (Fig. 7 and Fig. 8). Therefore, we will discuss their results together as one.

Our offline smoothing technique provides significant improvements to the results of the $\mathcal{B}$ solutions. When considering the geometric mean of the entire benchmark set on both scenarios (i and ii), our smoothing strategy improves the EDP results of the $\mathcal{B}$ solutions by 5% and 37% on *Intel24* and *Intel88*, respectively. As a result, our smoothing technique makes $\mathcal{B}$ a EDP optimization near to the upper bound on all applications, scenarios and machines. As a result, our smoothing technique makes the solution near the optimization upper-bound on all applications, scenarios, and machines. The most significant optimization cases are MG.B and SP.B when running with the best thread and data mapping (scenario ii) on *Intel88* (Fig. 8-ii): our technique optimizes the EDP results of $\mathcal{B}$ by 93% e 89% (MG.B and SP.B, respectively).

The optimization results are thanks to the reduction in thread count changes. To illustrate how our smoothing-based strategy reduces the thread count changes, Fig. 9 also draws

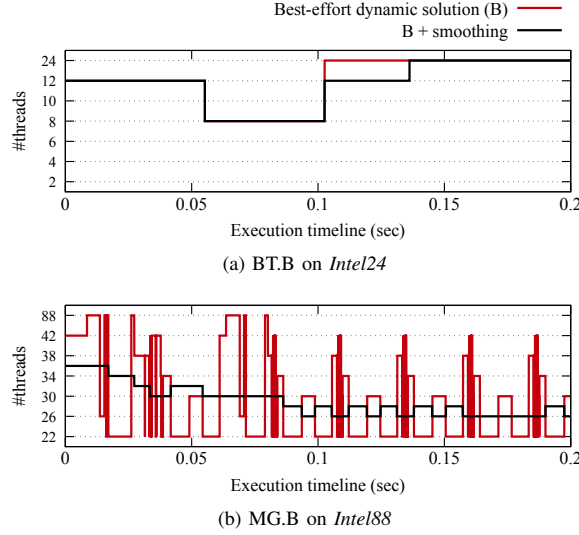


Fig. 9. Thread count (y-axis) used during the first 0.2 seconds of MG.B and BT.B benchmarks execution (x-axis) when they run with $\mathcal{B}$ solution with the best thread and data mapping (scenario ii)

the configurations after smoothing (black line). Notice that the changes reduce dramatically for MG.B (Fig. 9b).

C. Online smoothing results

Figures 7 and 8 also present the results of our online smoothing algorithms – **Online WMA-right-aligned** and **Online WMA-centered**. In these experiments, our strategy smooths the number of threads to run each parallel region as the application executes. However, $\mathcal{B}$ – which is the input for our algorithms – is still prior known. That is, the results presented in Fig. 7 and Fig. 8 have no learning overhead for finding $\mathcal{B}$. Therefore, the difference between the results of our online and offline strategies comes from the cost of the online smoothing process (collecting, storing, and analyzing data; predicting when it comes to the *centered* variation; and averages the values).

Our results show that our online smoothing algorithms have a low overhead. They deliver results only 3.5% and 4.6% different from our offline ones on *Intel24* (Fig. 7) and *Intel88* (Fig. 8) when considering the overall geometric mean of the entire benchmark set, scenarios (i and ii), and algorithm variations.

Evaluation of the region prediction. Our online WMA-centered variation has to predict the right side of the window. Table III presents, for each benchmark, the accuracy of our prediction strategy. More precisely, it shows the accuracy of predicting the next parallel region given the current and right-before one. We have used history vectors of 15 enters each, which is good enough for the benchmarks of our experiments (all of them have at most 15 parallel regions). Notice that our online WMA-centered variation has presented similar results as the offline one, which knows the whole time series. For applications with more parallel regions, the history vector can be larger, or one can use a hash function for the index.

TABLE III
ONLINE WMA-centered – REGION PREDICTION ACCURACY

App	BT.B	CG.B	FFT	FT.C	JA	LU.B	MG.B	PO	SP.B
Accuracy	98.5%	95.2%	100%	85.7%	99.8%	96.1%	83.5%	99.6%	99.5%

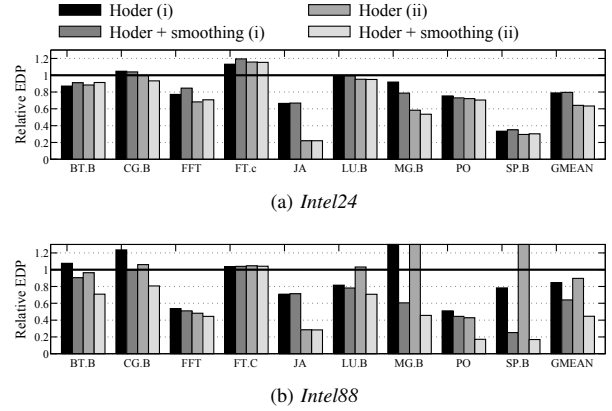


Fig. 10. EDP results of a state-of-the-art online DCT technique with our online smoothing strategy.

VII. CASE STUDY

For the experiments of the previous sections, we had selected $\mathcal{B}$, a best-effort dynamic solution, through an offline approach. This section shows that one can use our strategy to optimize DCT approaches that learn $\mathcal{B}$ at runtime. For that, we use the library Hoder [7], a state-of-the-art online learning DCT technique, as a case study. We use Hoder’s version that optimizes EDP by adapting only thread count configurations.³ In its learning phase, Hoder uses a Fibonacci-based algorithm to converge to a solution $\mathcal{B}$.

In the first execution of each parallel region j , our library collects the execution time and stores it into q_j . Also, as the application executes, whenever a parallel region executes, it stores the history $\mathcal{P}$. However, since the online learning techniques know $\mathcal{B}$ only after finishing the learning phase, our technique only starts smoothing the number of threads in the stable phase.

We execute Hoder without and with our smooth strategy (Hoder+smoothing) considering the two scenarios: execution with (i) the default and (ii) the best thread and data mapping policy. Fig. 10 shows the EDP results for the entire benchmark set along with their geometric mean (gmean) considering the two multicore systems. Due to space limitation, we present only the results of our WMA-right-aligned algorithm, but the two variations have presented similar results. Results are normalized by the baseline (values lower than 1 are better).

Our smoothing strategy improves Hoder’s EDP results by 22% when considering the overall geometric mean (entire benchmark set and both scenarios on the two machines). Notice that our strategy is primarily essential to take full advantage of optimization opportunities on NUMA systems with large thread count search space (on mean, it optimizes by

³Hoder also has a version that optimizes DVFS too. However, DVFS is not the focus of this work.

50% the Hoder's EDP results when considering the executions on Intel88 with the best thread and data mapping).

VIII. CONCLUSION

This paper proposes a smoothing-based strategy to further improve the optimization results of any DCT technique. Our strategy smooths the thread count changes alleviating the switching overheads, which is generated by DCT when changing the number of threads during application execution. Experiments on two multicore systems with nine well-known benchmarks show that our smoothing technique improves EDP results of offline and online state-of-the-art DCT techniques by up to 93% and 89% (overall mean of 22%), respectively.

ACKNOWLEDGMENT

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001, the Fundação de Amparo à Pesquisa do Estado do RS (FAPERGS) and the Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq). Some experiments in this work used the PCAD infrastructure, <http://gppd-hpc.inf.ufrgs.br>, at INF/UFRGS.

REFERENCES

- [1] M. M. Waldrop, "The chips are down for moore's law," *Nature News*, vol. 530, no. 7589, p. 144, 2016.
- [2] N. Jones, "How to stop data centres from gobbling up the world's electricity," *Nature*, vol. 561, no. 7722, pp. 163–167, 2018.
- [3] M. A. Suleman, M. K. Qureshi, and Y. N. Patt, "Feedback-driven threading: power-efficient and high-performance execution of multi-threaded workloads on cmps," *ACM Sigplan Notices*, vol. 43, no. 3, pp. 277–286, 2008.
- [4] J. Lee, H. Wu, M. Ravichandran, and N. Clark, "Thread tailor: dynamically weaving threads together for efficient, adaptive parallel applications," in *Proceedings of the 37th annual international symposium on Computer architecture*, 2010, pp. 270–279.
- [5] W. Wang, J. W. Davidson, and M. L. Soffa, "Predicting the memory bandwidth and optimal core allocations for multi-threaded applications on large-scale numa machines," in *IEEE International Symposium on High Performance Computer Architecture*. IEEE, 2016, pp. 419–431.
- [6] M. Popov, A. Jimborean, and D. Black-Schaffer, "Efficient thread/page/parallelism autotuning for numa systems," in *Proceedings of the ACM International Conference on Supercomputing*. ACM, 2019, pp. 342–353.
- [7] J. Schwarzrock, C. C. de Oliveira, M. Ritt, A. F. Lorenzon, and A. C. S. Beck, "A runtime and non-intrusive approach to optimize edp by tuning threads and cpu frequency for openmp applications," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 7, pp. 1713–1724, 2020.
- [8] G. Chadha, S. Mahlke, and S. Narayanasamy, "When less is more (limo): controlled parallelism for improved efficiency," in *Proceedings of the 2012 international conference on Compilers, architectures and synthesis for embedded systems*, 2012, pp. 141–150.
- [9] A. F. Lorenzon and A. C. S. Beck Filho, *Parallel computing hits the power wall: principles, challenges, and a survey of solutions*. Springer Nature, 2019.
- [10] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber *et al.*, "The nas parallel benchmarks summary and preliminary results," in *Proceedings of the 1991 ACM/IEEE conference on Supercomputing*. IEEE, 1991, pp. 158–165.
- [11] J. Li and J. F. Martinez, "Dynamic power-performance adaptation of parallel computation on chip multiprocessors," in *The Twelfth International Symposium on High-Performance Computer Architecture*, 2006. IEEE, 2006, pp. 77–87.
- [12] M. Curtis-Maury, J. Dzierwa, C. D. Antonopoulos, and D. S. Nikolopoulos, "Online power-performance adaptation of multithreaded programs using hardware event-based prediction," in *Proceedings of the 20th annual international conference on Supercomputing*, 2006, pp. 157–166.
- [13] M. Curtis-Maury, F. Blagojevic, C. D. Antonopoulos, and D. S. Nikolopoulos, "Prediction-based power-performance adaptation of multithreaded scientific codes," *IEEE Transactions on Parallel and Distributed Systems*, vol. 19, no. 10, pp. 1396–1410, 2008.
- [14] D. Li, B. R. de Supinski, M. Schulz, K. Cameron, and D. S. Nikolopoulos, "Hybrid mpi/openmp power-aware computing," in *IEEE International Symposium on Parallel & Distributed Processing*. IEEE, 2010, pp. 1–12.
- [15] S. Sridharan, G. Gupta, and G. S. Sohi, "Adaptive, efficient, parallel execution of parallel programs," in *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2014, pp. 169–180.
- [16] F. Alessi, P. Thoman, G. Georgakoudis, T. Fahringer, and D. S. Nikolopoulos, "Application-level energy awareness for openmp," in *International Workshop on OpenMP*. Springer, 2015, pp. 219–232.
- [17] D. De Sensi, M. Torquati, and M. Danelutto, "A reconfiguration algorithm for power-aware parallel applications," *ACM Transactions on Architecture and Code Optimization*, vol. 13, no. 4, pp. 1–25, 2016.
- [18] A. F. Lorenzon, C. C. De Oliveira, J. D. Souza, and A. C. S. Beck, "Aurora: Seamless optimization of openmp applications," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 5, pp. 1007–1021, 2018.
- [19] A. Savitzky and M. J. Golay, "Smoothing and differentiation of data by simplified least squares procedures," *Analytical chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964.
- [20] K. K. Pusukuri, R. Gupta, and L. N. Bhuyan, "Thread reinforcer: Dynamically determining number of threads via os level monitoring," in *2011 IEEE International Symposium on Workload Characterization*. IEEE, 2011, pp. 116–125.
- [21] D. De Sensi, "Predicting performance and power consumption of parallel applications," in *Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*. IEEE, 2016, pp. 200–207.
- [22] J. Schwarzrock, M. G. Jordan, G. Korol, C. C. d. Oliveira, A. F. Lorenzon, M. Beck Rutzig, and A. C. S. Beck, "Dynamic concurrency throttling on numa systems and data migration impacts," *Design Automation for Embedded Systems*, vol. 25, no. 2, pp. 135–160, 2021.
- [23] M. R. Garey and D. S. Johnson, *Computers and intractability*. freeman San Francisco, 1979, vol. 174.
- [24] T.-Y. Yeh and Y. N. Patt, "Two-level adaptive training branch prediction," in *Proceedings of the 24th annual international symposium on Microarchitecture*, 1991, pp. 51–61.
- [25] S. Seo, G. Jo, and J. Lee, "Performance characterization of the nas parallel benchmarks in opencl," in *2011 IEEE international symposium on workload characterization (IISWC)*. IEEE, 2011, pp. 137–148.
- [26] W. Petersen and P. Arbenz, *Introduction to Parallel Computing: A practical guide with examples in C*, ser. Oxford Texts in Applied and Engineering Mathematics. OUP Oxford, 2004.
- [27] M. Quinn, *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill Higher Education, 2004.
- [28] J. Schwarzrock, H. M. G. d. A. Rocha, A. F. Lorenzon, and A. C. S. Beck, "Effective exploration of thread throttling and thread/page mapping on numa systems," in *2020 IEEE 22st International Conference on High Performance Computing and Communications*. IEEE, 2020.
- [29] P. Radojković, P. M. Carpenter, M. Moreto, V. Čakarević, J. Verdu, A. Pajuelo, F. J. Cazorla, M. Nemirovsky, and M. Valero, "Thread assignment in multicore/multithreaded processors: a statistical approach," *IEEE Transactions on Computers*, vol. 65, no. 1, pp. 256–269, 2015.
- [30] A. E. Eichenberger, C. Terboven, M. Wong, and D. an Mey, "The design of openmp thread affinity," in *International Workshop on OpenMP*. Springer, 2012, pp. 15–28.
- [31] A. Kleen. (2004) An numa api for linux. [Online]. Available: <http://andikleen.de/numaapi3.pdf>
- [32] M. Diener, E. H. Cruz, M. A. Alves, P. O. Navaux, and I. Koren, "Affinity-based thread and data mapping in shared memory systems," *ACM Computing Surveys*, vol. 49, no. 4, pp. 1–38, 2016.
- [33] M. Hähnel, B. Döbel, M. Völpl, and H. Härtig, "Measuring energy consumption for short code paths using RAPL," *SIGMETRICS Perform. Eval. Rev.*, vol. 40, no. 3, pp. 13–17, 2012.